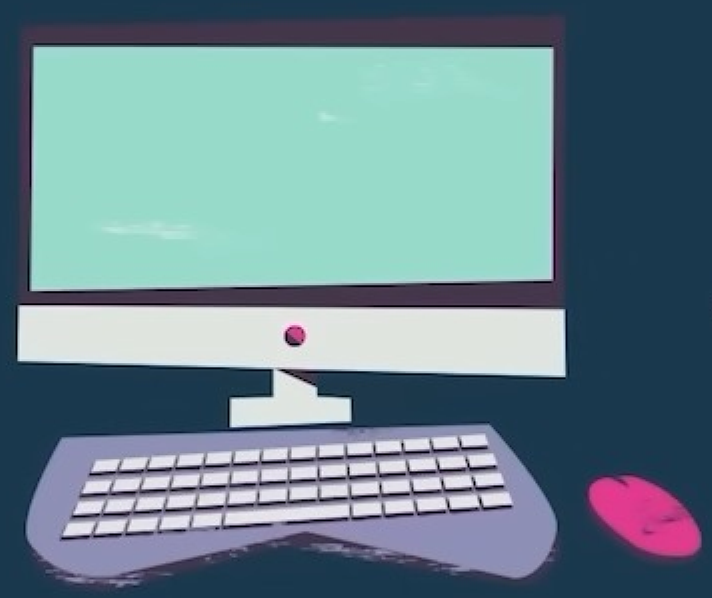

Input Output
variables
data types
operators
Introduction to Python
APNA COLLEGE

Programming



Machine ← **Translator** ← Code

(Compiler / Interpreter)

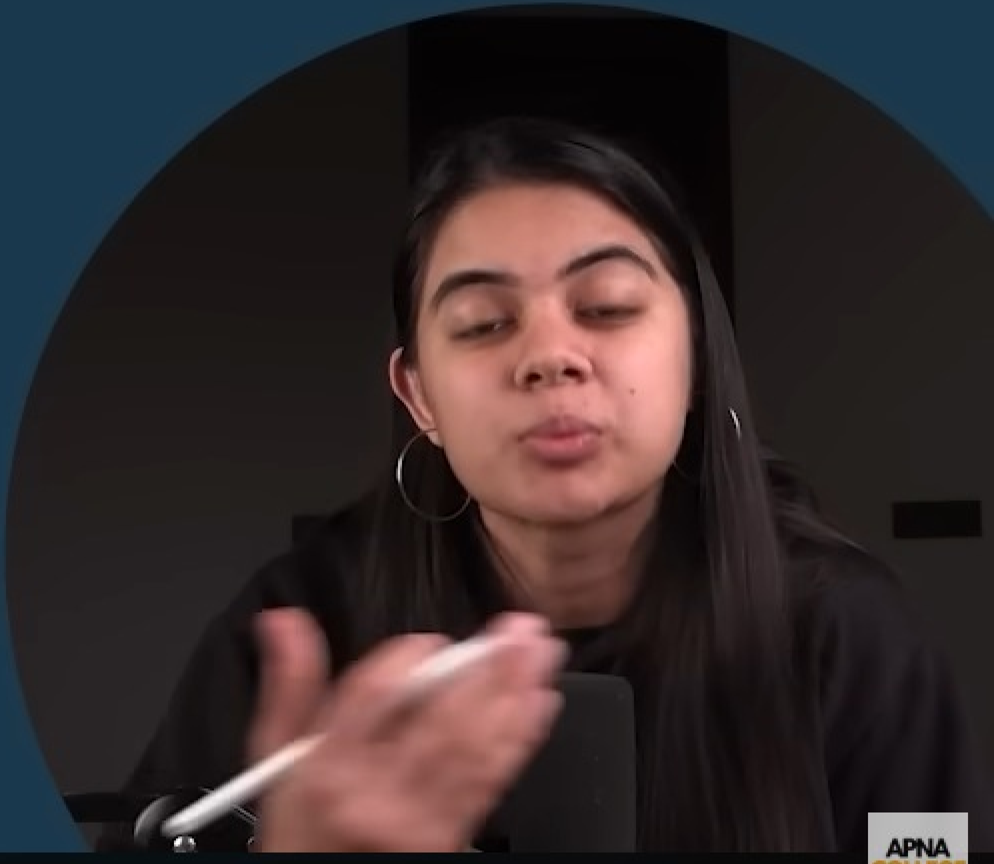
What is Python?

- Python is simple & easy
- Free & Open Source
- High Level Language
- Developed by Guido van Rossum
- Portable

What is Python?

DS — ML/AI
Web Dev — websites (Django)
Games

- Python is simple & easy
- Free & Open Source
- High Level Language
- Developed by Guido van Rossum
- Portable



FirstProgram.py

Users > shradhakhapra > Python > FirstProgram.py

```python
print("Hello World")
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.p
```

Our First Program

print("~~Hello World~~")

Apna College

function

अ आ

Python Character Set

- Letters - A to Z, a to z
- Digits - 0 to 9
- Special Symbols - + - * / etc.
- Whitespaces - Blank Space, tab, carriage return, newline, formfeed
- Other characters - Python can process all ASCII and Unicode characters as part of data or literals

Users > shradhakhapra > Python > FirstProgram.py

```python
print("Shradha is my name.", "My age is 23.")
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/First
Shradha is my name.
My age is 23.
```

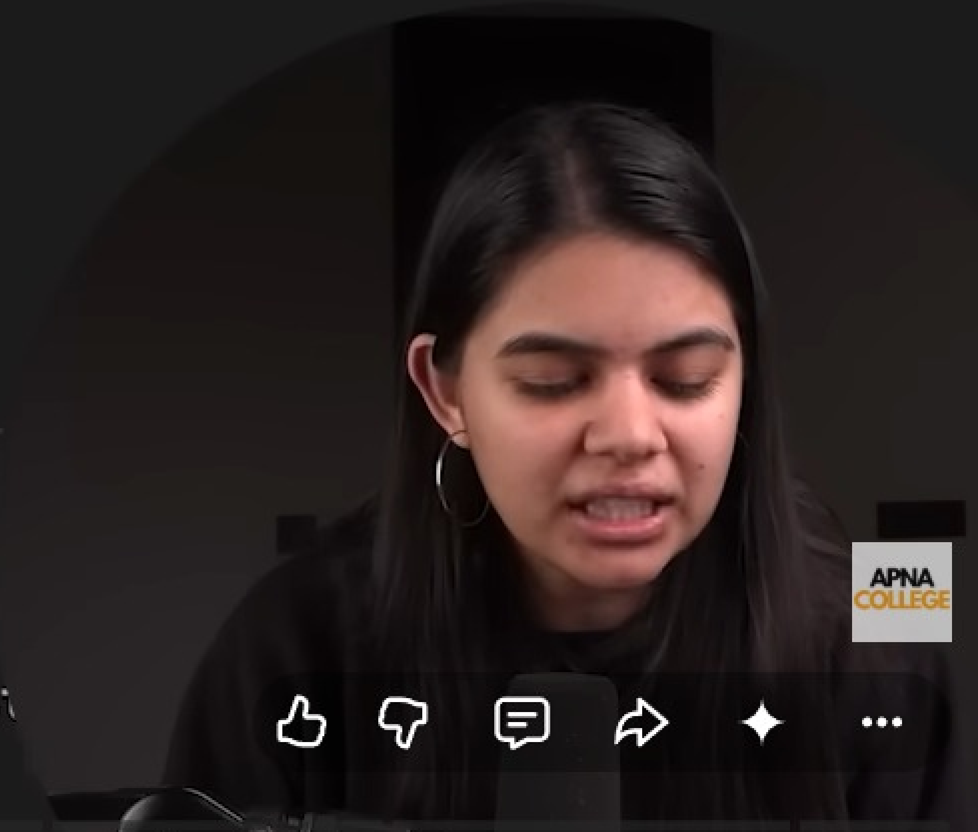
Python Tutorial for Beginners - Full Course (with Notes & Practice Questions)

Users > shradhakhapra > Python > FirstProgram.py

```python
print(23)
print(35+23)
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL

```
23
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/Firs
23
35
```



22:39 / 10:24:50 Variables and Data Types >

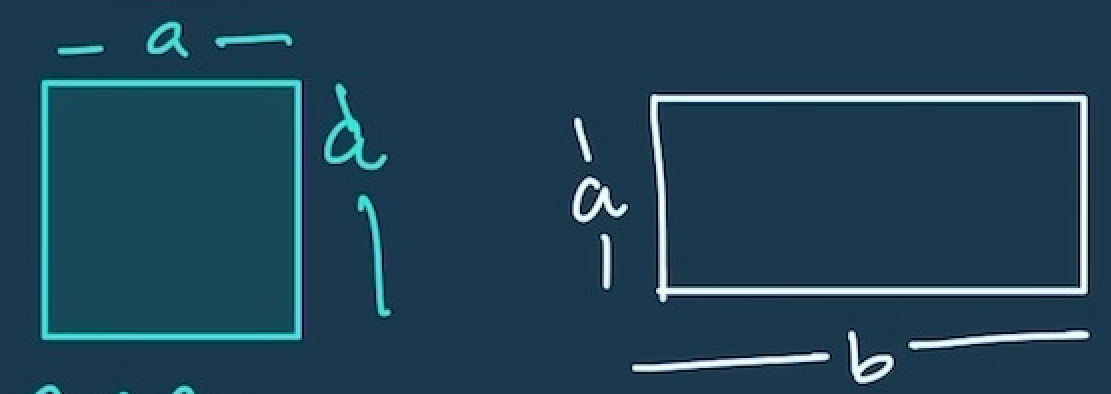
Variables

A variable is a name given to a memory location in a program.

```
name = "Shradha"
age = 23
price = 25.99
```

var = value

Variables



$a * a$

$a * b$

$2 * (a+b)$

A variable is a name given to a memory location in a program.

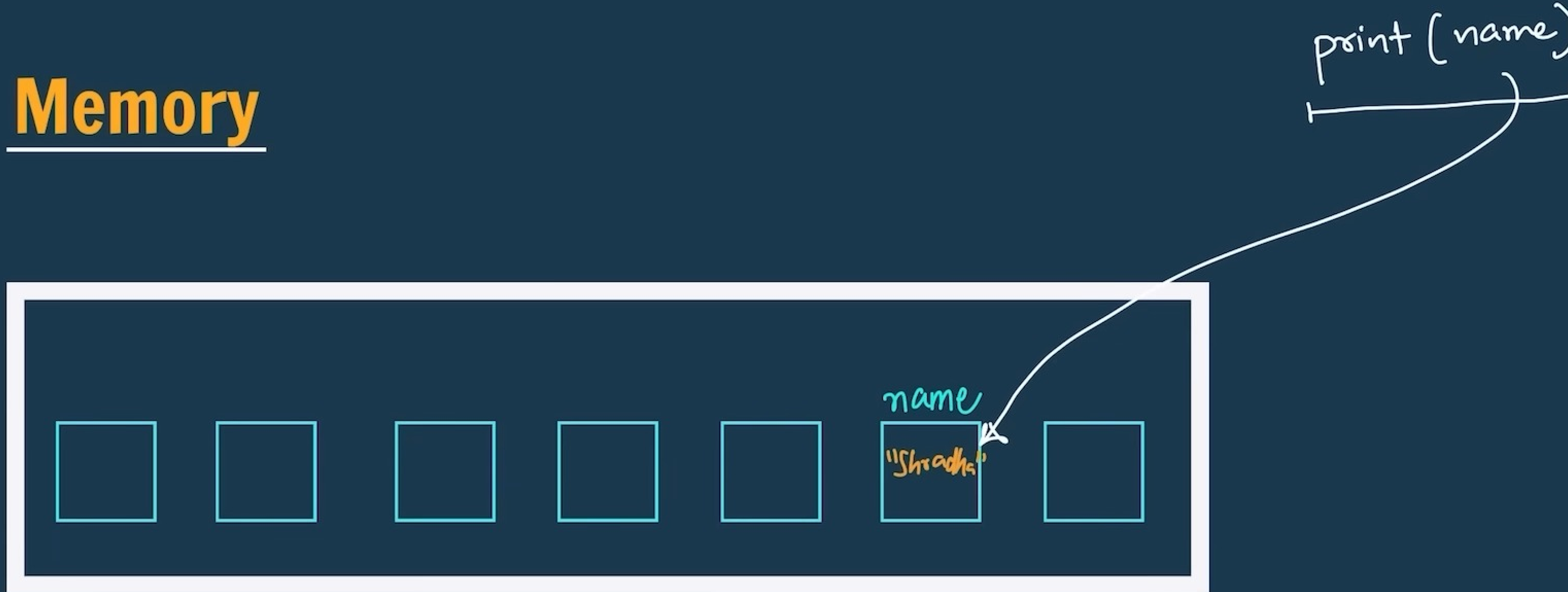
var   value

name = "Shradha"

age = ~~23~~ ~~24~~ 26

price = 25.99

Memory




```
name = "Shradha"
age = 23
price = 25.99
```

```python
name = "Shradha"
age = 23
price = 25.99

print(price)
```

DEBUG CONSOLE   PROBLEMS   OUTPUT   TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/FirstPro
name
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/First
age
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/Firs
23
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/Firs
Shradha
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/Firs
25.99
shradhakhapra@Shradhas-Air ~ %
```

Users > shradhakhapra > Python > FirstProgram.py > ...

```python
name = "Shradha"
age = 23
price = 25.99

print("my name is : ", name)
print("my age is : ", age)

```

DEBUG CONSOLE   PROBLEMS   OUTPUT   TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/First
my name is :  Shradha
my age is :  23
shradhakhapra@Shradhas-Air ~ % 
```

Users > shradhakhapra > Python > FirstProgram.py > age

```python
name = "Shradha"
age = 25
price = 25.99

age2 = age

print(age2)
```

DEBUG CONSOLE   PROBLEMS   OUTPUT   TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/First
25
shradhakhapra@Shradhas-Air ~ %
```

Rules for Identifiers

names

1. Identifiers can be combination of uppercase and lowercase letters, digits or an underscore(_). So **myVariable**, **variable_1**, **variable_for_print** all are valid python identifiers.
2. An Identifier can not start with digit. So while **variable1** is valid, **1variable** is not valid.
3. We can't use special symbols like !,#,@,%,$ etc in our Identifier.
4. Identifier can be of any length.

Rules for Identifiers

names
variable
functions

this is my python variable

1. Identifiers can be combination of uppercase and lowercase letters, digits or an underscore(_). So **myVariable**, **variable_1**, **variable_for_print** all are valid python identifiers.
2. An Identifier can not start with digit. So while **variable1** is valid, **1variable** is not valid.
3. We can't use special symbols like !,#,@,%,$ etc in our Identifier.
4. Identifier can be of any length.

a-z , A-Z
0-9 , _

1variable ✗
variable1 ✓

Users > shradhakhapra > Python > FirstProgram.py > ...

```python
name = "Shradha"
age = 25
price = 25.99

print(type(name))
print(type(age))
print(type(price))
```

DEBUG CONSOLE   PROBLEMS   OUTPUT   TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Python/First
<class 'str'>
<class 'int'>
<class 'float'>
shradhakhapra@Shradhas-Air ~ %
```

Data Types

- Integers
- String
- Float
- Boolean
- None

Data Types

- **Integers** +ve, -ve, 0 (25, -25, 0) int
- **String** "shradha" "Hello" ' ' " " ''' '''
- **Float**
- **Boolean**
- **None**

T/F

Data Types

- Integers   +ve , -ve , 0   (25, -25, 0)   int
- String   "shradha" "Hello"
- Float   3.99   2.5   9.0
- Boolean → True   False
- None

Users > shradhakhapra > Python > FirstProgram.py > ...

```python
age = 23
old = False
a = None
print(type(old))
print(type(a))
```

DEBUG CONSOLE   PROBLEMS   OUTPUT   TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Pyt
<class 'bool'>
<class 'NoneType'>
shradhakhapra@Shradhas-Air ~ %
```

Users > shradhakhapra > Python > FirstProgram.py > ...

```python
age = 23
old = True
a = None
print(type(old))
print(type(a))
```

DEBUG CONSOLE PROBLEMS OUTPUT TERMINAL

```
/usr/local/bin/python3 /Users/shradhakhapra/Python/FirstProgram.py
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Pyt
<class 'bool'>
<class 'NoneType'>
shradhakhapra@Shradhas-Air ~ % /usr/local/bin/python3 /Users/shradhakhapra/Pyt
<class 'bool'>
<class 'NoneType'>
shradhakhapra@Shradhas-Air ~ %
```

Keywords

Keywords are reserved words in python.

and	else	in	return
as	except	is	True
assert	finally	lambda	try
break	False	nonlocal	with
class	for	None	while
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

Print **Sum**

$a = 2$

$b = 5$

$sum = a + b$

Print Sum

```
a = 2
b = 5
sum = a + b
print (sum)
```

program

Types of Tokens

Punctuators

Punctuators are symbols to organize sentence structure in programming.

( ), { }, @, [ ], # etc.

-=, +=, /=, *=, //=, = etc.

Types of Tokens

$a * b + c$

$(a * b) + c$

Punctuators

Punctuators are symbols to organize sentence structure in programming.

name = "Shradha"

age = 23

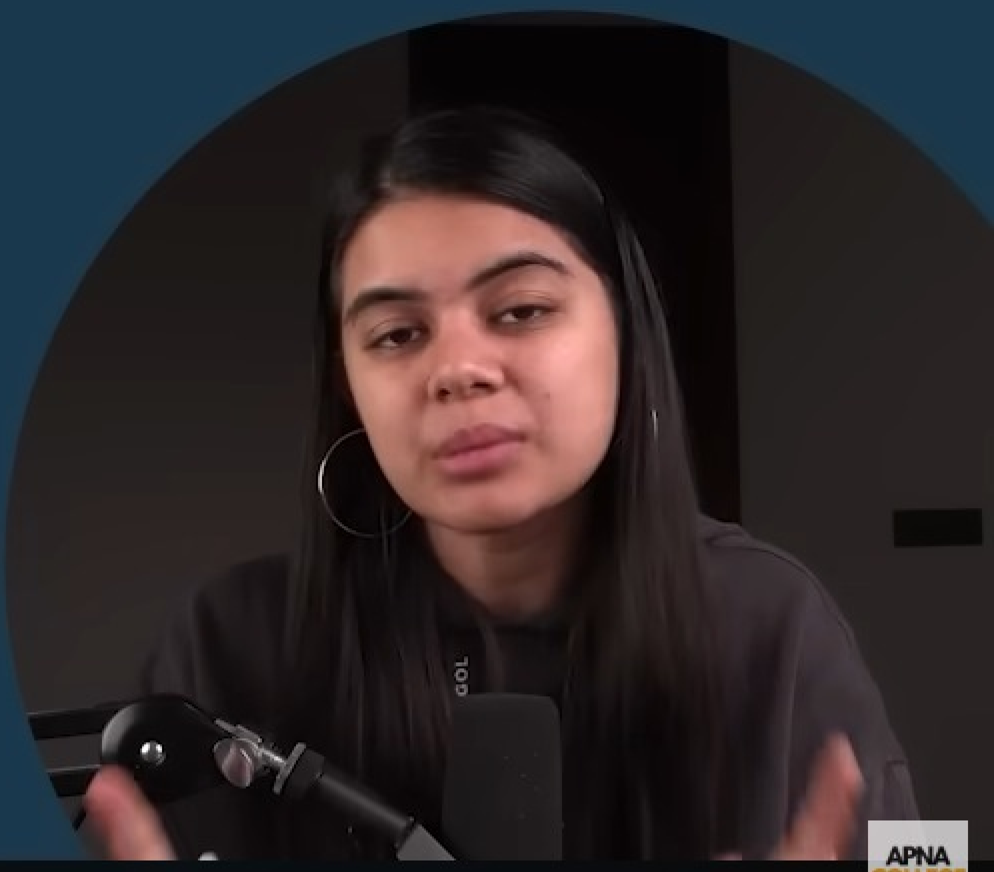
( ), { }, @, [ ], # etc.

-=, +=, /=, *=, //=, := etc.

Typed language

implicit — python

explicit — C++, java



Expression Execution

2 — numbers (int)

"2" — string

- String & Numeric values can operate together with * ← repeat

A = 2
B = 3

```
A,B=2,3
Txt="@"   ← str
print(2*Txt*3)
```

6

```
@@@@@@
```

@

- String & String can operate with +

A = "2"
B = 3

```
A,B="2",3
Txt="@"
print((A+Txt)*B)
```

"2" + "@"   "2@"

```
2@2@2@
```

1 2 3

Expression Execution

$2 + \underbrace{3 * 4}$

$2 + 12 = 14$

$* > +$

$\underset{int}{10} * \underset{float}{5.0} = \overset{float}{50.0}$

$\cancel{50}$

- **Numeric values can operate with all arithmetic operators**

A=2, B=3, C=4

```
A,B=2,3
C=4
print(A+B*C)
```

```
14
```

- **Arithmetic expression with Integer and float will result in float**

A=10, B=5.0

```
A,B=10,5.0
C=A*B
print(C)
```

```
50.0
```

Expression Execution

A = 1
B = 2

C = 1/2 = 0.5

- Result of division operator with two integers will be float

```
A,B=1,2
C=A/B
print(C)
```

```
0.5
```

//

- Integer division with float and int will give int displayed as float

A = 1.5
B = 3
C = 1.5//3
~~0.5~~ 0

```
A,B=1.5,3
C=A//B
print(C, A/B)
```

```
0.0 0.5
```

0.7 → 0   0.9999 → 0   1.2 → 1

floor(number) → closest

0.1 → 0
5.2 → 5
7.99 → 7
11.2 → 11
-5.2 → -6
5.0 → 5
2 —

Expression Execution

floor gives closest integer, which is lesser than or equal to the float value

Result of (A // B) is same as floor(A / B)

```
A,B=12,5
C=A//B
print(C)
```

```
2
```

```
A,B=-12,5
C=A//B
print(C)
```

```
-3
```

```
A,B=12,-5
C=A//B
print(C)
```

```
-3
```

Expression Execution

floor(number) → closest

0.1 → 0
5.2 → 5
7.99 → 7
11.2 → 11
-5.2 → -6
5.0 → 5
2 → 2

floor gives closest integer, which is lesser than or equal to the float value

Result of (A // B) is same as floor(A / B)

```python
A,B=12,5
C=A//B
print(C)
```

```
2
```

C = 12 // 5   (12/5)
2.4 → 2

(2)

```python
A,B=-12,5
C=A//B
print(C)
```

```
-3
```

A = -12 , B = 5
C = -12 // 5
C = -3

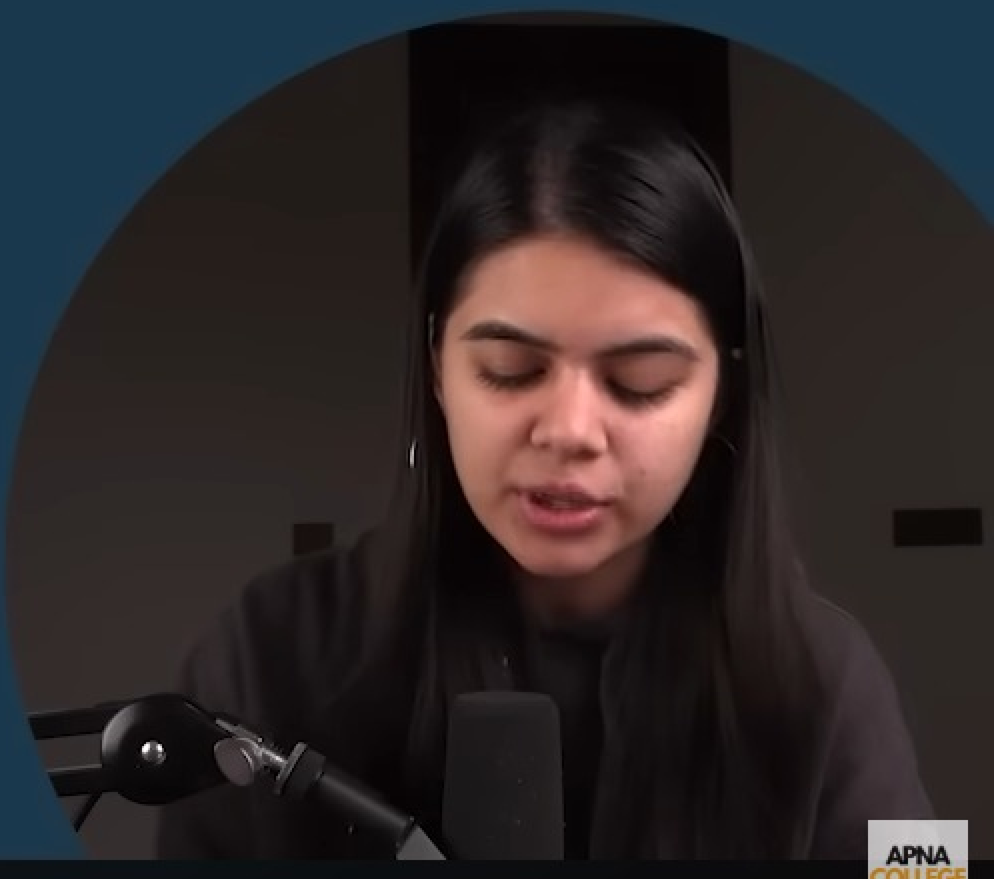
(-12 / 5)
-2.4
-3

(3)

```python
A,B=12,-5
C=A//B
print(C)
```

```
-3
```

C = 12 // -5   (12 / -5)
-2.4
-3



Expression Execution

Remainder is negative when denominator is negative

$\frac{n}{d}$: $\frac{+}{+} \rightarrow +$, $\frac{-}{-} \rightarrow +$, $\frac{+}{-} \rightarrow -ve$, $\frac{-}{+} \rightarrow +$

```python
A,B=-5,2
C=A%B
print(C)
```

```
1
```

$A = -5 \quad B = 2$

$C = \left(-5/2\right)$ −ve / +ve

$2 \overline{)\,5\,(}\,2$
-4
$\overline{1}$

```python
A,B=5,2
C=A%B
print(C)
```

```
1
```

$A = 5$
$B = 2$

$C = \left(5/2\right) \rightarrow 1$

```python
A,B=5,-2
C=A%B
print(C)
```

```
-1
```

$A = 5$
$B = -2$

$\left(\frac{5}{-2}\right) \rightarrow (-1)$

Comments in Python

#single line comment

""" This is
a multi-line
comment """

Shradha's code
#
#

```
#print("Hello")
print("World")
```

won't be printed
will be printed